

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1**Implementation of a Single Layer Perceptron for Binary Classification****1. Objective**

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. The experiment covers the perceptron learning algorithm, activation functions, exploratory data analysis, feature preprocessing, learning behaviour visualization, model evaluation, and learning-rate analysis using the Banknote Authentication Dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output. The Single Layer Perceptron is a supervised learning algorithm for binary classification and can represent a linear decision boundary.

Mathematical Formulation

The weighted sum is

$$z = \mathbf{w}^T \mathbf{x} + b,$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ is the weight vector, b is the bias, and z is the linear combination.

The prediction is

$$\hat{y} = f(z).$$

For a training example, the perceptron learning rule updates the parameters as

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x},$$

$$b_{new} = b_{old} + \eta(y - \hat{y}),$$

where η is the learning rate.

Perceptron Learning Process

The implementation initializes the weights to zero and the bias to zero. For each sample, the weighted sum is computed, the step function produces a binary prediction, and the weights and bias are updated whenever the prediction differs from the target. The number of mistakes is recorded after every epoch for analysis.

4. Activation Functions

An activation function determines the output of a neuron from its weighted input. The supplied laboratory manual specifies that this experiment uses only the Step Activation Function; differentiable activations are discussed for comparison and later deep-learning applications.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The step function produces a binary output and is appropriate for the classical perceptron. In the supplied implementation, the checks $f(8)$, $f(-6)$ and $f(0)$ produced 1, 0 and 1, respectively.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: The actual classifier in this experiment uses only the Step Activation Function.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link: <https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Features: Variance, Skewness, Curtosis and Entropy.

Target class: 0 – Authentic Banknote; 1 – Forged Banknote.

The notebook confirms a dataset shape of (1372, 5) including the four input features and the target column. All four features and the target contain 1372 non-null observations.

6. Experimental Procedure

Task 1: Dataset Exploration

The dataset was loaded into a Pandas DataFrame with the feature names variance, skewness, curtosis, entropy, and class. The first five records were displayed. The dataset dimensions were (1372, 5) and no missing values were found.

	Variance	Skewness	Curtosis	Entropy	Class
Count	1372	1372	1372	1372	1372
Mean	0.433735	1.922353	1.397627	-1.191657	0.444606
Std. Dev.	2.842763	5.869047	4.310030	2.101013	0.497103
Minimum	-7.042100	-13.773100	-5.286100	-8.548200	0
Maximum	6.824800	12.951600	17.927400	2.449500	1

Task 2: Exploratory Data Analysis

The required EDA consisted of feature histograms, a correlation heatmap, a scatter plot and box-plots. These plots are reproduced from the supplied notebook outputs.

Feature Histograms

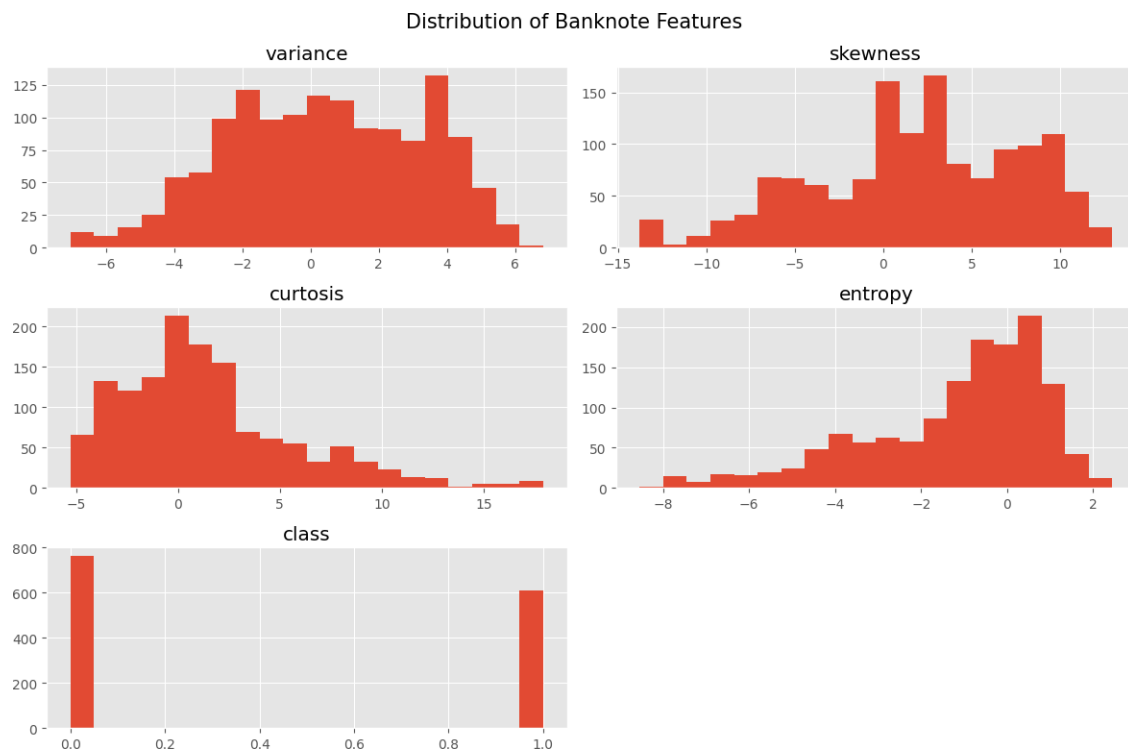


Figure 1: Distribution of Banknote Features

Interpretation: The four numerical features have different distributions and ranges. Curtosis and entropy show noticeable skewness and the plots also indicate observations in the tails, supporting the use of feature standardization before training.

Correlation Heatmap

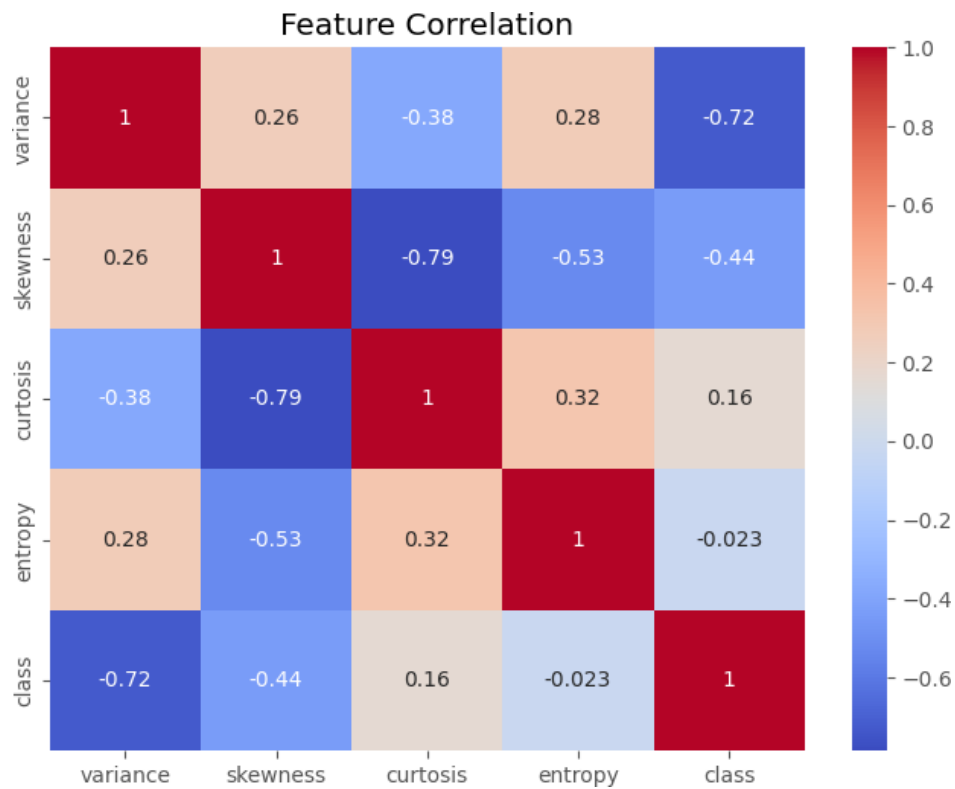


Figure 2: Feature Correlation Heatmap

Interpretation: Variance has a strong negative correlation with the class label (approximately -0.72), while skewness is also negatively correlated with class (approximately -0.44). Skewness and kurtosis show a strong negative relationship (approximately -0.79), indicating that the features are not independent.

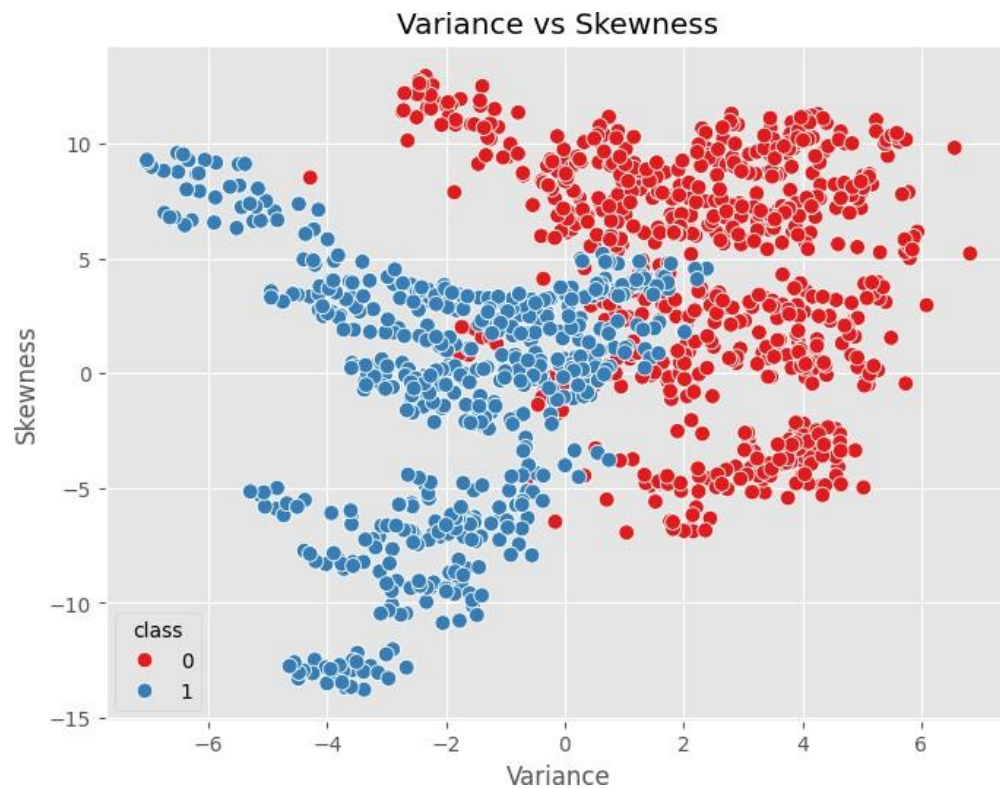
Scatter Plot

Figure 3: Variance versus Skewness by Class

Interpretation: The two classes form visibly different regions in the variance–skewness plane, although some overlap is present. The separation indicates that a linear classifier can obtain high predictive performance using the available features.

Boxplots

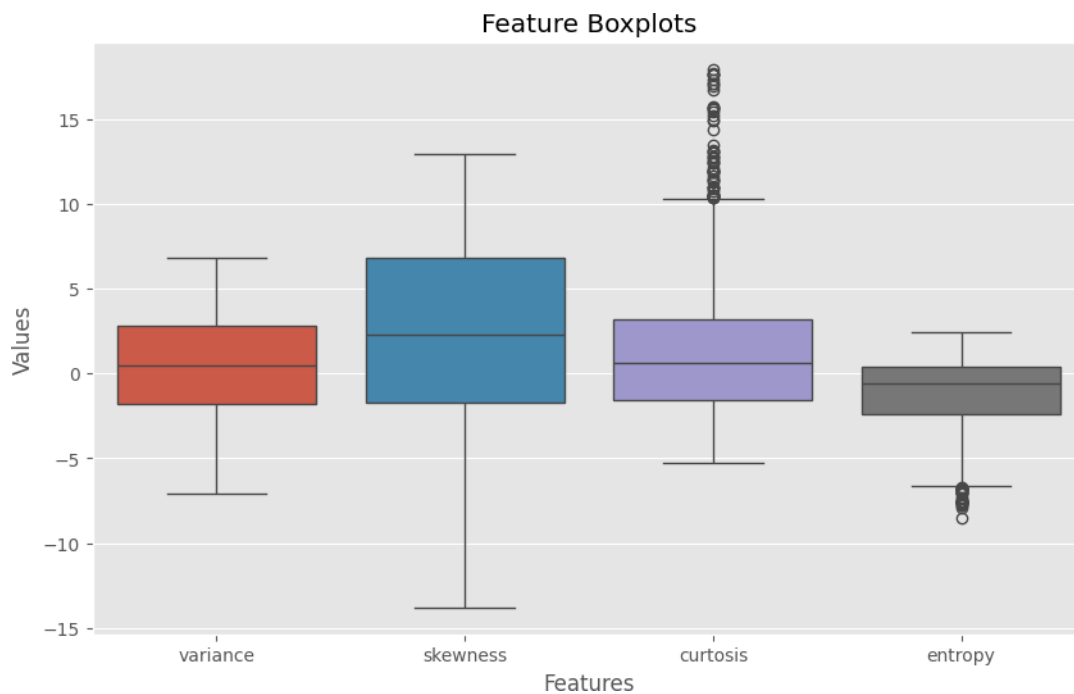


Figure 4: Feature Boxplots

Interpretation: The boxplots reveal different spreads and several outlying observations, particularly for kurtosis and entropy. Standardization is therefore useful to place the features on comparable scales before applying the perceptron update rule.

Task 3: Data Preprocessing

The class column was separated from the four numerical input features. The data was divided into training and testing sets using an 80%/20% split, with `random_state=42` and shuffling enabled. `StandardScaler` was fitted on the training data and then applied to both training and testing data. The resulting shapes were:

- Training set: 1097 samples $\times$ 4 features.
- Testing set: 275 samples $\times$ 4 features.

Task 4: Perceptron Implementation

A class named `PerceptronClassifier` was implemented from scratch. It contains weight initialization, bias initialization, a step activation function, single-sample forward prediction, the perceptron learning rule, prediction for a set of samples, and histories for errors, weights and bias.

The initial parameters were

$$\mathbf{w} = [0, 0, 0, 0], \quad b = 0.$$

The weighted sum for each sample is computed as $\mathbf{x} \cdot \mathbf{w} + b$, followed by the step activation. When an error occurs, the implementation applies the update rule given in Section 3.

Task 5: Model Training

The final experiment used learning rate $\eta = 0.01$ and 50 epochs. The first epoch produced 47 misclassified samples. The number of errors generally decreased over training, reaching a minimum of 11 errors at epoch 36, although the error count fluctuated because the perceptron performs sequential updates on individual samples.



Figure 5: Training Error versus Epoch

Interpretation: The training error decreases substantially from the initial value of 47 and fluctuates at lower levels as training progresses. The fluctuations are expected from online perceptron updates; the curve indicates that the learned parameters have entered a lower-error region rather than decreasing monotonically.

Task 6: Model Evaluation

The trained perceptron was evaluated on the 275-sample test set using accuracy, precision, recall, F1-score and a confusion matrix.

Metric	Value
Accuracy	0.981818 = 98.18%
Precision	1.000000 = 100.00%
Recall	0.960630 = 96.06%
F1-score	0.979920 = 97.99%

The confusion matrix was

	0	1
0	148	0
1	5	122

Thus, there were 148 true negatives, 0 false positives, 5 false negatives and 122 true positives under the displayed class-label convention.

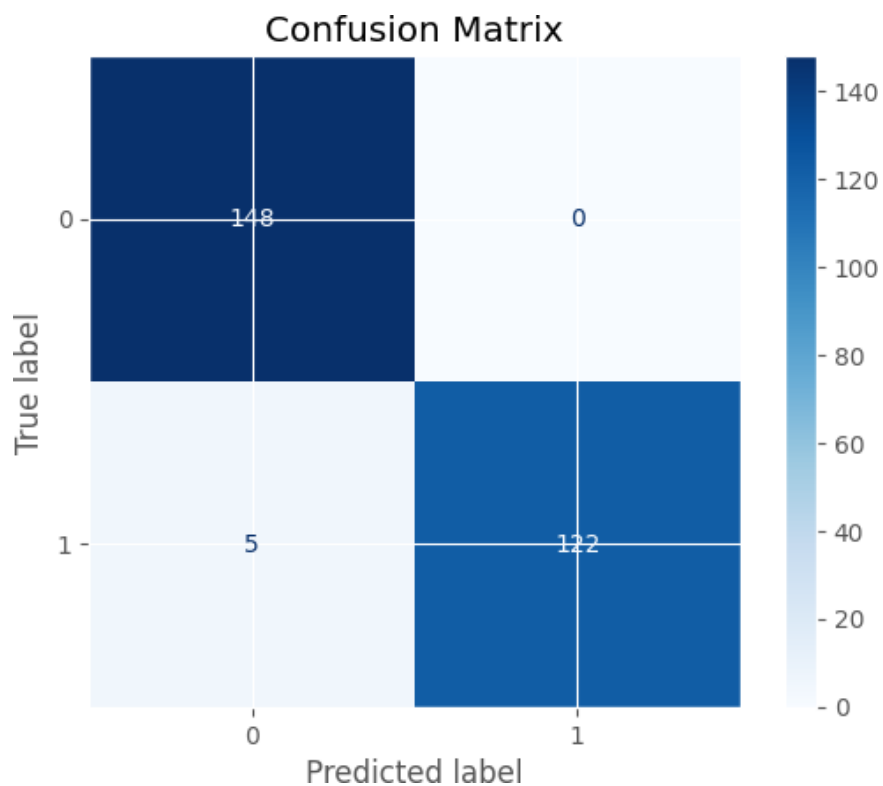


Figure 6: Confusion Matrix on the Test Set

Interpretation: The classifier correctly predicts 148 samples of class 0 and 122 samples of class 1. There are no false positives and five false negatives, which explains the perfect precision and the slightly lower recall.

Weight Evolution

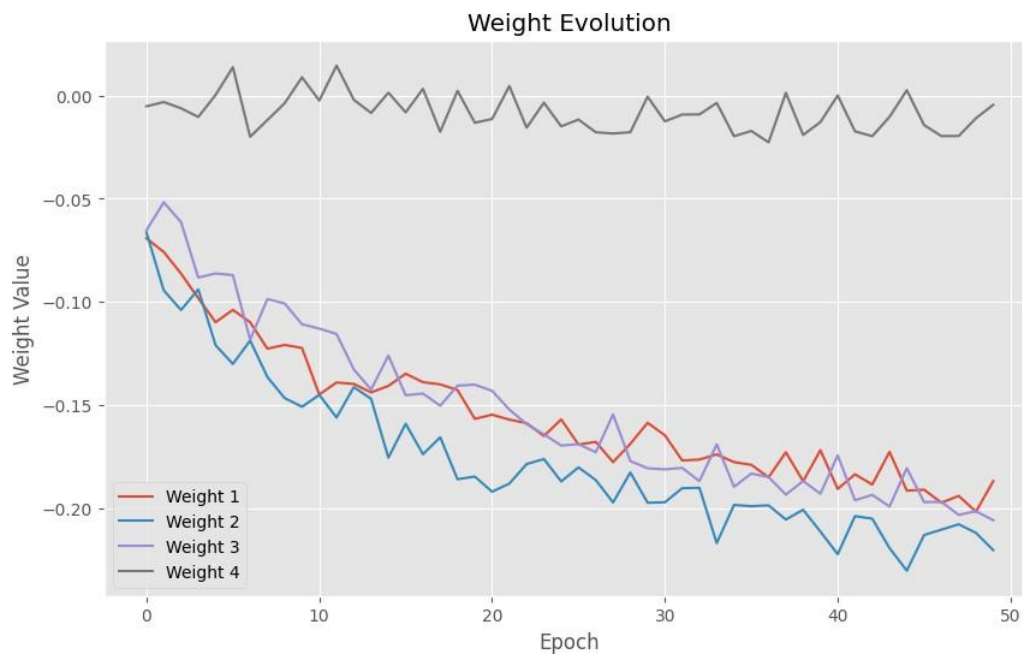


Figure 7: Evolution of the Four Learned Weights

Interpretation: The weights change rapidly during the early epochs and then vary within a relatively narrow region. The final values are approximately $[-0.18684, -0.22027, -0.20577, -0.00448]$, showing that the first three standardized features contribute more strongly to the learned linear decision function than the fourth feature.

Bias Evolution

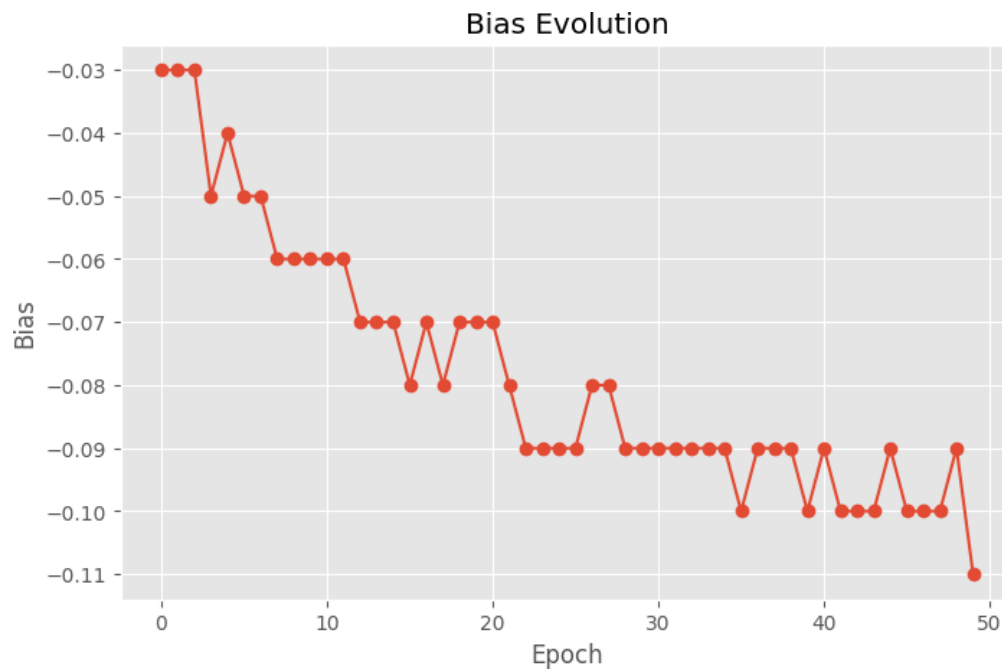


Figure 8: Evolution of the Bias Term

Interpretation: The bias begins near -0.03 and changes through discrete increments of the learning rate as classification mistakes occur. It reaches approximately -0.11 at epoch 50, showing how the intercept is adjusted together with the weights to shift the decision boundary.

Learning Rate Comparison

The manual specifies learning rates 0.001, 0.01 and 0.1. The supplied notebook trained separate perceptrons for all three values for 50 epochs and plotted their error histories.

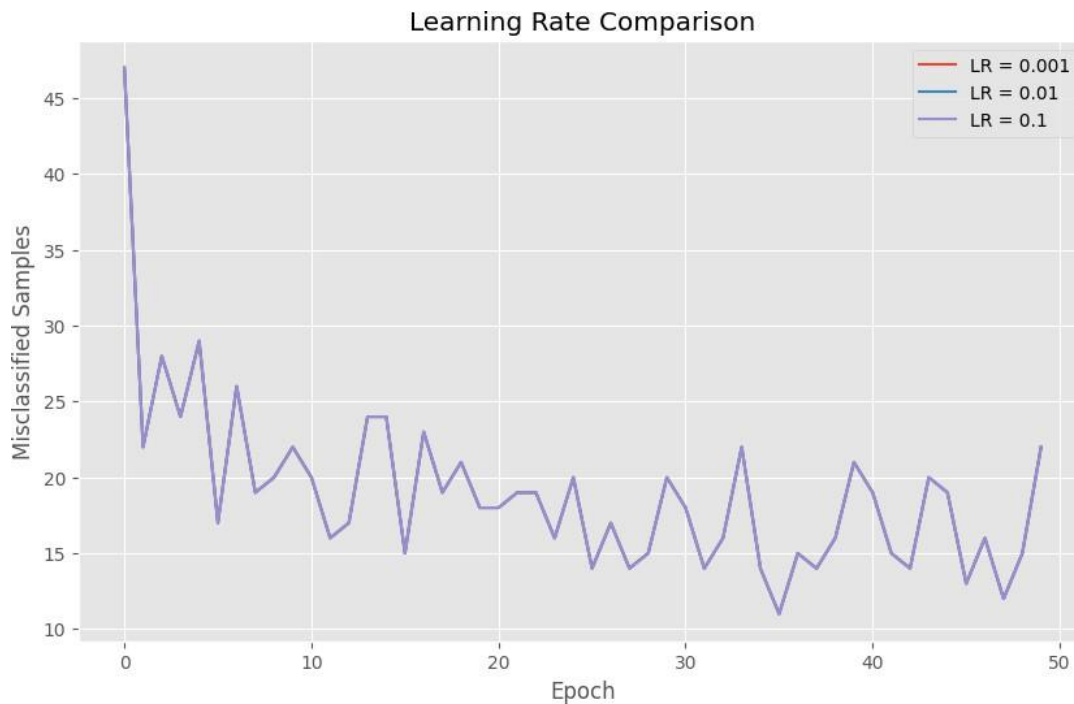


Figure 9: Learning Rate Comparison

Interpretation: In the supplied implementation, the three learning rates produce the same displayed sequence of misclassification counts across epochs. The classification path can remain identical because changing the learning rate scales the parameter updates while preserving their direction; the resulting parameter magnitudes differ, but the sign-based decisions may follow the same sequence.

7. Mandatory Plots and Interpretation Summary

Plot	Purpose	Observed Inference
Feature Histograms	Distribution Analysis	Different feature ranges and skewed tails are visible; standardization is appropriate.
Correlation Heatmap	Feature Relationship	Strong relationships exist, especially skewness–kurtosis and variance–class.
Scatter Plot	Class Distribution	The two classes occupy substantially different regions, supporting high linear-classifier performance.

Training Error vs Epoch	Learning Behaviour	Errors decrease from 47 initially and fluctuate at lower values later in training.
Weight Evolution	Parameter Analysis	Weights stabilize within a lower range after early updates.
Bias Evolution	Parameter Analysis	Bias changes incrementally as classification errors trigger updates.
Confusion Matrix	Performance Evaluation	148 TN, 0 FP, 5 FN and 122 TP are obtained.
Learning Rate Comparison	Hyperparameter Study	The displayed error histories coincide for 0.001, 0.01 and 0.1 in this run.
Decision Boundary	Optional Visualization	Not generated in the supplied notebook; it is optional in the laboratory manual.

8. Performance Tables

Training Summary

Dataset Size	1372 samples, 4 input features + 1 target
Train/Test Split	80% / 20%
Learning Rate	0.01
Epochs	50
Final Weights	$[-0.18684277, -0.22027144, -0.20576685, -0.00448154]$
Final Bias	-0.11
Accuracy	98.18%
Precision	100.00%
Recall	96.06%
F1-score	97.99%

Epoch-wise Learning

The laboratory manual asks for epoch, errors, Weight 1, Weight 2 and bias. The complete 50-epoch parameter history is represented by the training plots, while selected checkpoints are tabulated below for readability.

Epoch	Errors	Weight 1	Weight 2	Bias
1	47	-0.06920	-0.06634	-0.03
5	29	-0.10985	-0.12095	-0.04
10	22	-0.12230	-0.15079	-0.06
15	24	-0.14069	-0.17547	-0.07
20	18	-0.15666	-0.18461	-0.07
25	20	-0.15693	-0.18698	-0.09
30	20	-0.15855	-0.19728	-0.09
35	14	-0.17762	-0.19837	-0.09
36	11	-0.17893	-0.19895	-0.10
40	21	-0.17184	-0.21129	-0.10
45	19	-0.19141	-0.23022	-0.09
46	13	-0.19098	-0.21294	-0.10
47	16	-0.19728	-0.21033	-0.10
48	12	-0.19399	-0.20774	-0.10
49	15	-0.20160	-0.21189	-0.09
50	22	-0.18684	-0.22027	-0.11

9. Additional Tasks

1. Step versus Sigmoid

The Step activation outputs only 0 or 1 and is non-differentiable at zero. The Sigmoid activation produces a continuous output in (0, 1) and has a useful derivative, making it suitable for gradient-based optimization. Consequently, the hard Step function is not used as the main activation in modern deep networks trained by backpropagation.

2. Comparison with Scikit-learn Perceptron

The supplied notebook implements the classifier from scratch and does not contain a Scikit-learn Perceptron training/evaluation cell. Therefore, no measured second-model result is claimed here. Conceptually, both methods learn a linear decision function, but the custom implementation exposes the individual parameter updates and epoch histories required for this experiment.

3. Learning Rates 0.001, 0.01 and 0.1

The supplied notebook explicitly evaluates all three requested learning rates. The resulting error histories are coincident in the generated comparison plot. Therefore, for this dataset, split, initialization and 50-epoch procedure, changing the learning rate changed update magnitudes without changing the displayed sequence of misclassified samples.

4. Why a Single Layer Perceptron Cannot Solve XOR

XOR is not linearly separable. A single perceptron can create only one linear decision boundary, whereas XOR requires a nonlinear separation of the four input combinations. A multilayer network with a hidden layer and nonlinear activation is therefore required to represent XOR.

5. Effect of Feature Normalization

The experiment standardizes the four numerical features using `StandardScaler` fitted only on the training data. This places features on comparable scales and prevents features with larger numerical magnitudes from dominating the perceptron updates. The supplied notebook does not include a separate unnormalized training run, so the report does not claim a measured before/after convergence improvement; the conclusion is based on the preprocessing used and the role of scaling in the update rule.

10. Experimental Results and Discussion

The from-scratch perceptron achieved 98.18% accuracy on the 275-sample test set. Precision reached 100%, meaning that none of the samples predicted as class 1 were false positives. Recall was 96.06% because five class-1 samples were misclassified as class 0. The resulting F1-score of 97.99% demonstrates a strong balance between precision and recall.

The EDA shows substantial feature relationships and different feature scales, while the scatter plot indicates useful class separation. Standardization followed by the perceptron learning rule produced a high-performing linear classifier. The training-error, weight and bias plots show the expected iterative adjustment of parameters rather than a smooth gradient-based optimization trajectory.

11. Conclusion

A Single Layer Perceptron was successfully implemented from scratch for binary classification of the Banknote Authentication Dataset. The experiment covered dataset exploration, EDA, standardization, zero initialization, Step activation, perceptron parameter updates, epoch-wise monitoring, model evaluation and learning-rate comparison. The final model achieved 98.18% accuracy and 97.99% F1-score on the test set, demonstrating that a linear perceptron can perform very well when the classes have substantial linear separability in the feature space.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.

5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.